Разделение прав пользователя и администратора.

В теге header внутри файла main.php подкаталога layouts каталога views создадим пустой массив для отображения элементов меню навигации.

```
$items = [];
```

В виджете меню навигации в параметре items, закомментируем код, и добавим созданный массив.

```
echo Nav::widget([
    'options' => ['class' => 'navbar-nav'],
    'items' => $items,
```

Теперь для каждой роли мы преобразуем меню навигации. Для гостя присутствуют ссылки на страницы Регистрация и Авторизация.

Если пользователь является гостем (user->isGuest), то создадим два новых элемента массива.

```
if(Yii::$app->user->isGuest) {
    $items[] = ['label' => 'Регистрация', 'url' => ['/user/create']];
    $items[] = ['label' => 'Авторизация', 'url' => ['/site/login']];
}
```

В противном случае гость является зарегистрированным пользователем, но у нас есть две роли: обычный пользователь и администратор.

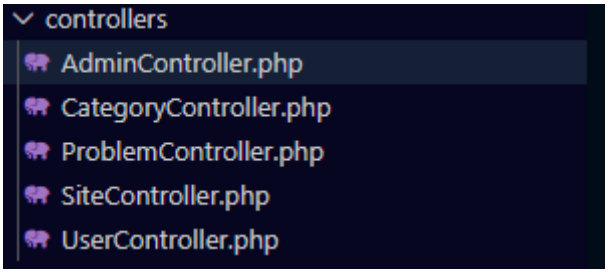
Если поле admin_user равно 1, то добавим ссылку на административную панель. Иначе добавим ссылку на личный кабинет.

```
else {
    if(Yii::$app->user->identity->admin_user == 1) {
        $items[] = ['label' => 'Административая панель', 'url' => ['/admin']];
    } else {
        $items[] = ['label' => 'Личный кабинет', 'url' => ['/user']];
    }
}
```

Для каждого зарегистрированного пользователя добавим выход из аккаунта.

```
$items[] = '<li class="nav-item">'
            . Html::beginForm(['/site/logout'])
            . Html::submitButton(
                'Выход (' . Yii::$app->user->identity->login_user . ')',
                ['class' => 'nav-link btn btn-link logout']
            )
            . Html::endForm()
            . '</li>';
```

Скопируем UserController и переименуем его в AdminController.



Меняем наименование класса модели.

```
class AdminController extends Controller
```

Стираем все функции кроме actionIndex.

```
<?php

namespace app\controllers;

use Yii;
use app\models\User;
use app\models\RegForm;
use app\models\UserSearch;
use yii\web\Controller;
use yii\web\NotFoundHttpException;
use yii\filters\VerbFilter;

/**
 * UserController implements the CRUD actions for User model.
 */
class AdminController extends Controller
{

    /**
     * Lists all User models.
     *
     * @return string
     */
    public function actionIndex()
    {
        $searchModel = new UserSearch();
        $dataProvider = $searchModel->search($this->request->queryParams);

        return $this->render('index', [
            'searchModel' => $searchModel,
            'dataProvider' => $dataProvider,
        ]);
    }


}
```

Создадим новое представление подкаталог admin. Скопируем в подкаталог файл index.php из подкаталога category.

```php
<?php

use app\models\Category;
use yii\helpers\Html;
use yii\helpers\Url;
use yii\grid\ActionColumn;
use yii\grid\GridView;

/** @var yii\web\View $this */
/** @var app\models\CategorySearch $searchModel */
/** @var yii\data\ActiveDataProvider $dataProvider */

$this->title = 'Административная панель';
$this->params['breadcrumbs'][] = $this->title;
?>
<div class="user-index">

    <h1><?= Html::encode($this->title) ?></h1>

     <p>
        <?= Html::a('Управление категории', ['/category'], ['class' => 'btn btn-success']) ?>
    </p>

    <?php // echo $this->render('_search', ['model' => $searchModel]); ?>




</div>
```

Но в теперь в админ панель можно попасть любому пользователю через строку поиска прописав /admin. Чтобы это избежать используется следующий метод.

Метод beforeAction() в Yii2, который запускается перед выполнением любого действия в контроллере. Он предназначен для предварительной проверки условий доступа к действиям контроллера и позволяет управлять правами доступа пользователей.

После вызывается родительский метод beforeAction() (так как это действие может быть перезаписано в дочерних контроллерах), и если родительский метод вернул false, выполнение прерывается, если действие прошло проверку, метод возвращает true.

```php
public function beforeAction($action)
{
    if(!parent::beforeAction($action)) {
        return false;
    }

    return true;
}
```

Добавим проверку гостевого режима и проверку наличия прав администратора, если хотя бы один из двух вариантов истинное выражение, то возвращаем пользователя на страницу авторизации.

```php
if(Yii::$app->user->isGuest || Yii::$app->user->identity->admin != 1) {
    $this->redirect('/site/login');
    return false;
}
```

Создадим контроллер для личного кабинета пользователя. Скопируем UserController.php и переименуем его в LkController.php.

LkController.php

```php
class LkController extends Controller
```

Добавим ссылку на личный кабинет в представление lk.

```php
$items[] = ['label' => 'Личный кабинет', 'url' => ['/lk']];
```

Создадим подкаталог lk внутри каталога views и скопируем index.php из подкаталога problems.

lk
index.php

В LkController найдем функцию actionIndex.

```php
public function actionIndex()
{
    $searchModel = new UserSearch();
    $dataProvider = $searchModel->search($this->request->queryParams);

    return $this->render('index', [
        'searchModel' => $searchModel,
        'dataProvider' => $dataProvider,
    ]);
}
```

Поменяем UserSearch() на модель ProblemSearch().

```
$searchModel = new ProblemSearch();
```

Импортируем модель Problem, и импортируем модель поиска ProblemSearch.

```
use Yii;
use app\models\User;
use app\models\Problem;
use app\models\RegForm;
use app\models\ProblemSearch;
use app\models\UserSearch;
use yii\web\Controller;
use yii\web\NotFoundHttpException;
use yii\filters\VerbFilter;
```

При переходе в личный кабинет отображаются все существующие заявки с проблемами.

Home / Problems

Problems

Create Problem

Showing **1-2** of **2** items.

#	Id Problem	Name Problem	Description Problem	Timestamp Problem	Id User	
1	1	Ямы на дороге	Емае, машина упала в яму!!!:(	2026-01-01 17:53:20	1	
2	2	Мусор	Мусорные контейнеры переполнились, и жители стали бросать мусор рядом, вонище редкосное, разбиритесь пж	2026-01-01 18:00:28	2	

Ограничим доступ к просмотру заявок для неавторизованных пользователей с помощью метода beforeAction(). Скопируем его в LkController и в условии оставим проверку гостевого режима.

```
public function beforeAction($action)
{

    if(Yii::$app->user->isGuest) {
        $this->redirect('/site/login');
        return false;
    }

    if(!parent::beforeAction($action)) {
        return false;
    }

    return true;
}
```

Откроем модель ProblemSearch.php каталога models.

Функция search() – данная функция позволяет гибко настраивать поисковые запросы, передавая произвольные параметры и дополнительные опции, такие как название формы, если это необходимо.

```
public function search($params, $formName = null)
{
    $query = Problem::find();

    // add conditions that should always apply here

    $dataProvider = new ActiveDataProvider([
        'query' => $query,
    ]);

    $this->load($params, $formName);

    if (!$this->validate()) {
        // uncomment the following line if you do not want to return any records when validation fails
        // $query->where('0=1');
        return $dataProvider;
    }

    // grid filtering conditions
    $query->andFilterWhere([
        'id_problem' => $this->id_problem,
        'timestamp_problem' => $this->timestamp_problem,
        'id_user' => $this->id_user,
        'id_category' => $this->id_category,
    ]);

    $query->andFilterWhere(['like', 'name_problem', $this->name_problem])
        ->andFilterWhere(['like', 'description_problem', $this->description_problem])
        ->andFilterWhere(['like', 'status_problem', $this->status_problem])
        ->andFilterWhere(['like', 'photo_before', $this->photo_before])
        ->andFilterWhere(['like', 'photo_after', $this->photo_after]);

    return $dataProvider;
}
```

Скопируем эту функцию ниже и переименуем ее в searchForUser(), и поменяем второй параметр на $id_user.

```
public function searchForUser($params, $id_user)
```

После условной конструкции с валидацией добавим $query->andWhere(['id_user' => $id_user]); такая конструкция применяется для дополнительной фильтрации результатов запроса по какому-то параметру. Это полезно, если вам нужно выбрать записи конкретного пользователя или ограничить доступ к ресурсам на уровне SQL-запросов.

```
$query->andWhere(['id_user' => $id_user]);
```

Удалим фильтрацию для идентификатора пользователя из массива andFilterWhere.

```
$query->andFilterWhere([
    'id_problem' => $this->id_problem,
    'timestamp_problem' => $this->timestamp_problem,

    'id_category' => $this->id_category,
]);
```

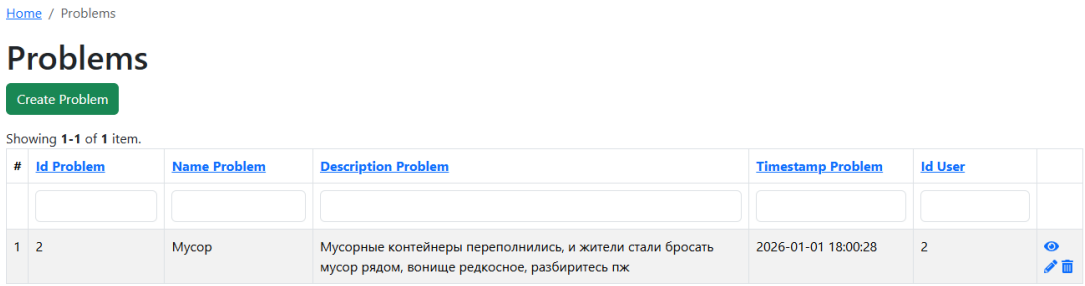
$query->orderBy(['timestamp_problem' => SORT_DESC]); – данная конструкция используется для сортировки в обратном хронологическом порядке.

```
$query->orderBy(['timestamp_problem' => SORT_DESC]);
```

Вернемся в LkController в функции actionIndex() поменяем функцию search на searchForUser.

```
$dataProvider = $searchModel->searchForUser($this->request->queryParams, Yii::$app->user->identity->id_user);
```

Таким образом мы отфильтровали заявки, чтобы каждый пользователь видел лишь свои заявки.



Отобразим лишь необходимые нам поля таблицы с заявками. Для этого в файле index.php подкаталога lk закомментируем ненужные нам поля и оставим только: временная метка, название и описание заявки, категория и статус заявки.

```php
<?= GridView::widget([
    'dataProvider' => $dataProvider,
    'filterModel' => $searchModel,
    'columns' => [
        // ['class' => 'yii\grid\SerialColumn'],

        // 'id_problem',
        'timestamp_problem',
        'name_problem',
        'description_problem:ntext',

        // 'id_user',
        'id_category',
        'status_problem',
        //'photo_before',
        //'photo_after',
        [
            'class' => ActionColumn::class Name(),
            'urlCreator' => function ($action, Problem $model, $key, $index, $column) {
                return Url::toRoute([$action, 'id_problem' => $model->id_problem]);
            }
        ],
    ],
]); ?>
```

Отобразим категории не с помощью идентификатора, а с помощью названия.

attribute — это свойство модели, соответствующее полю в таблице БД.

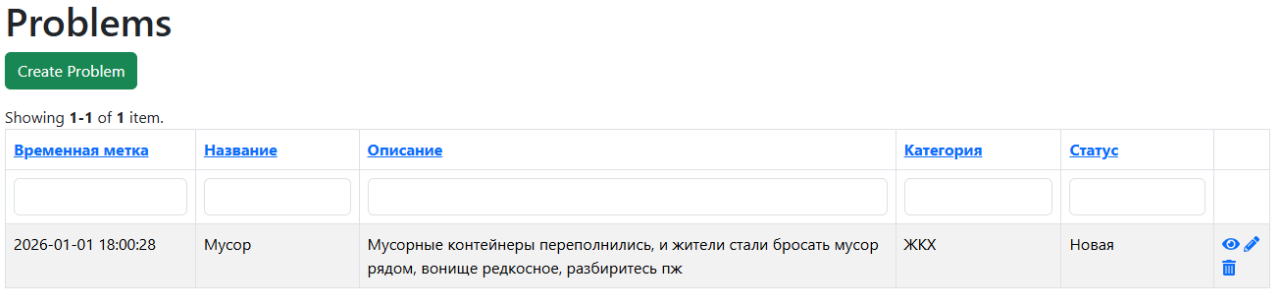
value — определяет, какое именно значение должно отображаться в ячейке соответствующего столбца. Вместо вывода ID категории используем выражение category.name_category, которое подразумевает получение названия категории из связанной сущности (например, из другой таблицы категорий). Такой подход часто встречается в отношениях многие-ко-многим или один-ко-многим.

```php
[
    'attribute' => 'id_category',
    'value' => 'category.name_category',
],
```

В модели Problems русифицируем название полей таблицы.

```php
public function attributeLabels()
{
    return [
        'id_problem' => 'Id Problem',
        'name_problem' => 'Название',
        'description_problem' => 'Описание',
        'timestamp_problem' => 'Временная метка',
        'id_user' => 'Id User',
        'id_category' => 'Категория',
        'status_problem' => 'Статус',
        'photo_before' => 'Photo Before',
        'photo_after' => 'Photo After',
    ];
}
```

Таким образом мы оформили вывод таблицы problems.



Оформим кнопки для действий, нам необходимы две операции просмотр и удаление. Внутри класса ActionColumn.

```php
'class' => ActionColumn::class.Name(),
'urlCreator' => function ($action, Problem $model, $key, $index, $column) {
    return Url::toRoute([$action, 'id_problem' => $model->id_problem]);
}
```

Теперь на странице отображается две кнопки для операции.

```
'template' => '{view} {delete}',
```

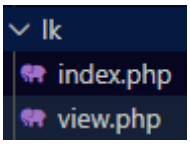
В LkController найдем функцию findModel(). Преобразуем ее для использования таблиц с проблемами.

Представленный код — это стандартный паттерн контроллера в Yii Framework, обеспечивающий проверку существования модели по уникальному идентификатору и обработку ситуации, когда объект не найден.

```php
protected function findModel($id_problem)
{
    if (($model = Problem::findOne(['id_problem' => $id_problem])) !== null) {
        return $model;
    }

    throw new NotFoundHttpException('The requested page does not exist.');
}
```

Скопируем в подкаталог lk файл view.php из подкаталога problem.



В LkController найдем функцию actionView(). Преобразуем ее для использования таблиц с проблемами.

```php
public function actionView($id_problem)
{
    return $this->render('view', [
        'model' => $this->findModel($id_problem),
    ]);
}
```

Таким образом страница для просмотра заявки реализована.

Home / Problems / 2

2

Update Delete

Id Problem	2
Название	Мусор
Описание	Мусорные контейнеры переполнились, и жители стали бросать мусор рядом, вонище редкосное, разбиритесь пж
Временная метка	2026-01-01 18:00:28
Id User	2
Категория	2
Статус	Новая
Photo Before	2-1.jpg
Photo After	*(not set)*